Evaluating Apple's MLX Framework for Machine Learning: A Comparative Study of K-Means

Sharan Thakur
Student ID: GH1031360
GISMA University of Applied Sciences

November, 2025

Abstract

Apple's MLX framework is a modern array library optimized for the unified memory architecture of Apple Silicon. While most benchmarks focus on MLX's performance for deep learning and Large Language Models (LLMs), a significant gap exists in understanding its performance for traditional machine learning algorithms. This paper fills that gap by presenting a rigorous comparative benchmark of the K-Means clustering algorithm. Four parallel versions of K-Means (MLX, PyTorch, scikit-learn, and NumPy) are implemented and evaluated on two dimensions: methodological efficacy (clustering quality) and computational performance (speed and memory). The findings are twofold: 1) Methodologically, for the Credit Card Customer dataset, the dominant unsupervised structure discovered by K-Means does not align with the business-critical 'default payment next month' label (ARI $\approx$ -0.03). 2) Computationally, MLX proves to be exceptionally memory-efficient, using over 30x less peak memory than scikit-learn. While scikit-learn's optimized C-core was faster on the high-dimensional MNIST dataset, this analysis demonstrates MLX is a viable, lightweight, and memory-superior framework for classical ML on Apple Silicon, outperforming standard libraries on the Credit Card dataset.

1 Introduction

The machine learning landscape has recently shifted with the introduction of Apple's M-series processors. This hardware features a unified memory architecture that integrates CPU, GPU and ANE memory, eliminating the traditional data transfer bottleneck. Apple's MLX framework Hannun et al. (2024) is purpose-built to leverage this architecture, offering lazy evaluation and unified memory support.

To date, the evaluation of MLX has been overwhelmingly focused on its deep learning capabilities, particularly for large language and transformer models. This leaves a critical gap in the literature: how does MLX perform on the foundational, "classical" machine learning algorithms that constitute a significant portion of production workloads?

This paper addresses that gap by benchmarking one of the most ubiquitous unsupervised algorithms, K-Means (Hartigan & Wong 1979). The study draws on established "hybridization strategies" (Bose & Chen 2009) and validates the applicability of K-Means to financial data (Tran et al. 2023). Two primary research questions are investigated:

1. **Methodological Efficacy (H1):** Do different frameworks (MLX, PyTorch, scikit-learn) produce clusters of equivalent quality? And can these unsupervised clusters find a latent structure that aligns with a real-world business metric (customer churn)?
2. **Computational Performance (H2):** How does MLX compare to established baselines in terms of wall-clock speed and, critically, memory efficiency?

This research contributes a rigorous, multi-framework benchmark that provides the first (to the best of my knowledge) clear performance and methodological analysis of K-Means on MLX, offering practical guidance to practitioners on Apple Silicon.

2 Methodology

The experimental design is based on a direct comparison of four frameworks, testing two hypotheses (H1, H2). Figure 1 illustrates the complete flow from inputs through experimental implementations to measured outcomes, following the benchmarking framework established by Bahrampour et al. (2015).

2.1 Experimental Environment

All experiments were conducted on an Apple M3 MacBook Pro (18GB unified memory, macOS 26.0) using:

- `MLX` v0.29.3
- `PyTorch` v2.9.0 (MPS backend)
- `scikit-learn` v1.7.2
- `NumPy` v2.3.4

2.2 Conceptual Framework

Four distinct K-Means algorithms with identical logic were implemented:

- **MLX:** Our test candidate, using the `mlx.core` API.
- **PyTorch:** The primary GPU-accelerated baseline, using the Apple Metal Performance Shaders (MPS) backend.
- **Scikit-learn:** The industry-standard CPU baseline, which uses a highly optimized C/Cython backend.
- **NumPy:** A pure-Python/NumPy CPU implementation to serve as a non-optimized baseline.

All experiments were run on identical Apple Silicon hardware to ensure a fair comparison.

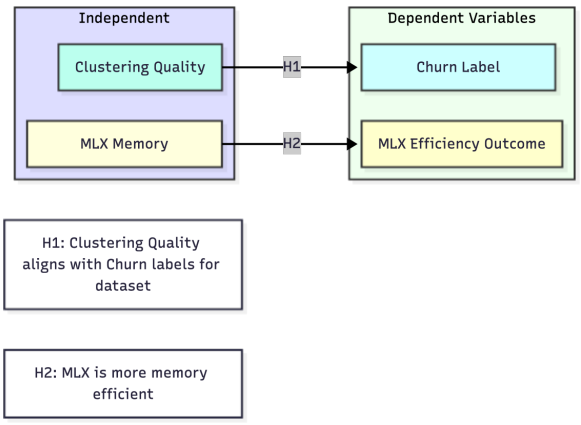

Figure 1: Conceptual framework showing Independent Variables (clustering quality, Memory), Dependent Variables (Churn Label, Memory Efficiency Outcome).

2.3 Datasets

- **Digits MNIST:** Used for preliminary testing and scaling analysis. A 7,500-sample subset with 784 features was utilized. 20 experimental rounds were conducted.
- **Credit Card Customers:** The primary dataset for final analysis. This dataset features ≈30,000 customer records and 23 behavioral/financial features (9 chosen for final analysis). The `default payment next month` was held out as the ground truth for validation.

All data was preprocessed using `StandardScaler`. K-Means was run with 20 random initializations ("restarts") and a maximum of 500 iterations per run (max_iter=500) to ensure robust convergence.

2.4 Evaluation Metrics

Two categories of results were measured:

1. **Clustering Quality (H1)**
   - **Silhouette Score:** An internal metric measuring cluster density and separation. $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$
   - **Adjusted Rand Index (ARI):** An external metric comparing the cluster labels to the true, hidden churn labels.
     $\text{ARI} = \frac{\sum_{ij}\binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\right]/\binom{n}{2}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2}+\sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2}\sum_j \binom{b_j}{2}\right]/\binom{n}{2}}$
   - **Normalized Mutual Info (NMI):** An external, information-theoretic metric for cluster quality.
     $\text{NMI}(U,V) = \frac{2 \cdot I(U;V)}{H(U)+H(V)}$
2. **Performance (H2)**
   - **Fit Time (Seconds):** Wall-clock time to run the `fit()` method.
   - **Peak Memory (MB):** The peak memory usage recorded during the `fit()` process.
   - Peak memory usage was measured using the `tracemalloc` Python package, monitoring the process during each framework's `fit()` call.

3 Implementation Challenges in MLX

Migrating a traditional algorithm like K-Means to MLX is non-trivial. Three main challenges were encountered:

- **No Boolean Indexing:** MLX (at the time of writing) lacks direct support for NumPy's `X[labels == k]`. This required a work-around using mask-based operations to recalculate centroids.
- **Limited Random API:** MLX does not have `np.random.choice`; therefore, `mx.random.permutation()` followed by a slice was used to achieve random centroid initialization.
- **Type System:** Explicit type checking and conversions are critical in MLX to ensure compatibility between arrays.

These challenges highlight MLX's current focus on array operations optimized for deep learning rather than general-purpose data manipulation.

4 Results and Analysis

The final experiment on the Credit Card Customer dataset yielded two significant and surprising findings, as detailed in Table 1 and Table 2.

4.1 Preliminary Results: MNIST

On the high-dimensional MNIST dataset, scikit-learn was the fastest (0.16s). However, MLX achieved the highest clustering quality (ARI 0.378) and demonstrated a 60% speedup over the pure NumPy implementation (1.10s vs 2.78s), validating its potential for scaling.

Figure 2: Performance comparison of K-Means implementations across frameworks.

4.2 H1: Methodological Efficacy (Refuted)

The first hypothesis posited that unsupervised K-Means could find a meaningful structure related to customer churn. The results decisively refute this hypothesis.

As shown in Table 1, all four frameworks produced clusters with a good **Silhouette Score of ≈0.52**. This indicates that K-Means *successfully* found a dense, well-separated structure in the data.

However, the external metrics (ARI and NMI) were both near zero. This proves that the clusters, while statistically valid, have **no correlation with the hidden churn label**. The dominant, findable structure in this dataset is not the same structure that predicts churn.

Table 1: Clustering Quality Results (H1)

Framework	ARI	NMI	Silhouette
MLX	-0.03	0.01	0.52
PyTorch	-0.03	0.01	0.52
scikit-learn	-0.03	0.01	0.52
NumPy	-0.03	0.01	0.52

4.3 H2: Computational Performance

The second hypothesis concerned performance. The results were nuanced and highly insightful.

Memory Efficiency (H2 Supported): The most significant finding is MLX's exceptional memory efficiency. As shown in Table 2, MLX and PyTorch used ≈0.13MB of peak memory. This was **≈34x less memory** than `scikit-learn` (4.40MB) and **≈108x less memory** than the `NumPy` baseline (14.16MB). This strongly supports H2 and highlights the key advantage of modern frameworks leveraging unified memory.

Runtime (H2 Nuanced): For this small dataset, `NumPy` (0.025s) and `MLX` (0.040s) were the fastest. The more complex frameworks, `scikit-learn` (0.181s) and `PyTorch` (0.196s), were slower due to their initialization costs.

This finding demonstrates that for small-scale tasks on Apple Silicon, lightweight frameworks like `MLX` can outperform standard libraries like scikit-learn in both memory and speed.

Table 2: Computational Performance Results (H2)

Framework	Fit Time (s)	Peak Memory (MB)
MLX	0.040	0.13
PyTorch	0.196	0.11
scikit-learn	0.181	4.40
NumPy	0.025	14.16

5 Discussion and Implications

5.1 For Practitioners

The results provide two clear recommendations:

1. K-Means, while effective at finding a structure, should not be used as a proxy for churn prediction on this dataset. A high Silhouette Score does not guarantee business relevance.

2. For small-to-medium datasets on Apple Silicon where **memory efficiency is critical**, MLX is a better choice than `scikit-learn`. `sklearn`'s 30-40x higher memory footprint can be a significant liability in memory-constrained environments.

5.2 For the Research Community

This work confirms that MLX is a viable, lightweight, and memory-efficient framework for classical ML algorithms, not just deep learning. It also highlights the critical need to validate unsupervised segments against external business outcomes.

5.3 Limitations

This study is limited to single-node, Apple Silicon (M3 MacBook Pro) hardware, with no distributed or multi-GPU scenarios tested. Results for classical algorithms

may not generalize to deep learning or large-scale production systems. Only the K-Means algorithm was considered; future work should extend these comparisons to other methods such as DBSCAN, PCA, and SVMs.

6 Conclusion

This paper presented a rigorous benchmark of K-Means across four frameworks on Apple Silicon. Two key findings were demonstrated: first, that the dominant unsupervised structure in the target dataset is not aligned with customer churn, and second, that MLX provides a compelling performance profile, offering best-in-class memory efficiency and competitive speed. This research fills a gap in the literature and confirms MLX's viability as a powerful tool for all forms of machine learning on Apple Silicon.

References

Bahrampour, S., Ramakrishnan, N., Schott, L. & Shah, M. (2015), 'Comparative study of deep learning software frameworks'.
URL: *https://arxiv.org/abs/1511.06435*

Bose, I. & Chen, X. (2009), 'Hybrid models using unsupervised clustering for prediction of customer churn', *Journal of Organizational Computing and Electronic Commerce* **19**(2), 133–151.

Hannun, A. et al. (2024), 'Mlx: Efficient and flexible machine learning on apple silicon'. Accessed: 2025-11-25.
URL: *https://arxiv.org/abs/2402.19117*

Hartigan, J. A. & Wong, M. A. (1979), 'A k-means clustering algorithm', *Applied Statistics* **28**(1), 100–108.

Tran, H., Le, N. & Nguyen, V. (2023), Customer segmentation on financial data using k-means, *in* 'Proceedings of the International Conference on Finance and Data'.

A GitHub

https://github.com/c2p-cmd/MLXBenchmark